JAVA DOCS

Multithreading & Concurrency in Java

Threads, Synchronization, Executors, and the java.util.concurrent Toolkit

Java Agentic AI — Knowledge Base Reference

1. Processes vs Threads

A **process** is an independent program in execution with its own memory space. A **thread** is a lightweight unit of execution within a process; all threads of a process share the same heap memory but each has its own stack, program counter, and local variables. Multithreading enables concurrent execution within a single process, improving CPU utilization and responsiveness.

2. Creating Threads

```
// Approach 1: extend Thread
class MyThread extends Thread {
    public void run() { System.out.println("Running in " + getName()); }
}
new MyThread().start();         // start() spawns a new OS thread; NEVER call run()
directly

// Approach 2: implement Runnable (preferred - allows extending other classes, decouples
task from thread)
Runnable task = () -> System.out.println("Running via Runnable");
new Thread(task).start();

// Approach 3: Callable + Future (can return a value / throw checked exceptions)
Callable<Integer> callable = () -> { Thread.sleep(100); return 42; };
```

Note: *Calling run() directly executes the code synchronously on the current thread -- no new thread is created. Only start() actually spawns a new thread of execution.*

3. Thread Lifecycle

State	Description
NEW	Thread object created but start() not yet called
RUNNABLE	Executing or eligible to be scheduled by the OS
BLOCKED	Waiting to acquire a monitor lock (synchronized block)
WAITING	Waiting indefinitely for another thread (wait(), join() with no timeout)
TIMED_WAITING	Waiting for a bounded time (sleep(ms), wait(ms), join(ms))
TERMINATED	run() has completed (normally or via exception)

4. Synchronization

Concurrent access to shared mutable state can cause **race conditions**. Java's built-in synchronized keyword ensures mutual exclusion using an intrinsic monitor lock associated with every object.

```
class Counter {
    private int count = 0;

    public synchronized void increment() { // method-level lock on 'this'
```

```
        count++;
    }

    public void incrementBlock() {
        synchronized (this) {                // block-level lock, same monitor
            count++;
        }
    }

    private static final Object LOCK = new Object();
    public void safeStaticOp() {
        synchronized (LOCK) { /* ... */ }     // custom lock object
    }
}
```

- A synchronized instance method locks on 'this'; a synchronized static method locks on the Class object.
- Only one thread can hold a given monitor at a time; others block until it's released.
- Locks in Java are **reentrant** — a thread already holding a lock can re-acquire it without deadlocking itself.

5. volatile Keyword

volatile guarantees **visibility** (every read sees the latest write from any thread, bypassing CPU caches) and prevents instruction reordering around it, but it does **not** provide atomicity for compound operations like increment (read-modify-write).

```
private volatile boolean running = true; // visibility guarantee across threads

public void stop() { running = false; }
public void loop() { while (running) { /* work */ } }
```

Note: *count++ on a volatile int is still NOT thread-safe -- it's three separate operations (read, increment, write) that can interleave. Use AtomicInteger for atomic compound operations instead.*

6. wait(), notify(), notifyAll()

These methods (defined on Object) coordinate threads waiting on a shared condition. They must be called from within a synchronized block on the same monitor, or an IllegalMonitorStateException is thrown.

```
class Buffer {
    private final Queue<Integer> queue = new LinkedList<>();
    private final int capacity = 5;

    synchronized void produce(int value) throws InterruptedException {
        while (queue.size() == capacity) wait(); // release lock, wait for space
        queue.add(value);
        notifyAll();                                // wake up waiting consumers
    }
```

```
    synchronized int consume() throws InterruptedException {
        while (queue.isEmpty()) wait();
        int val = queue.poll();
        notifyAll();
        return val;
    }
}
```

Note: *Always call wait() inside a while loop (not if) to guard against spurious wakeups and to re-check the condition after being notified.*

7. Executor Framework

Manually managing raw Thread objects doesn't scale. The Executor framework (java.util.concurrent) manages a pool of reusable worker threads, decoupling task submission from thread lifecycle.

```
ExecutorService executor = Executors.newFixedThreadPool(4);

Future<Integer> future = executor.submit(() -> {
    Thread.sleep(100);
    return 42;
});

Integer result = future.get();         // blocks until the task completes
executor.shutdown();                     // graceful shutdown - lets running tasks finish

// Common factory methods:
Executors.newFixedThreadPool(n);        // fixed-size pool
Executors.newCachedThreadPool();        // grows/shrinks, reuses idle threads
Executors.newSingleThreadExecutor();    // serial execution, one worker
Executors.newScheduledThreadPool(n);    // delayed / periodic tasks
```

Note: *Prefer constructing ThreadPoolExecutor directly with explicit core/max pool size and a bounded queue in production code -- the Executors factory methods can create pools with unbounded queues or unbounded thread counts, risking OutOfMemoryError under load.*

8. CompletableFuture

```
CompletableFuture<Integer> cf = CompletableFuture.supplyAsync(() -> compute())
    .thenApply(result -> result * 2)
    .thenAccept(System.out::println)
    .exceptionally(ex -> { System.out.println("Error: " + ex); return null; });

CompletableFuture<Void> combined = CompletableFuture.allOf(future1, future2, future3);
```

9. Locks and Atomic Variables

```
// Explicit Lock - more flexible than synchronized (tryLock, timed lock, interruptible)
private final ReentrantLock lock = new ReentrantLock();

public void update() {
    lock.lock();
```

```
    try {
        // critical section
    } finally {
        lock.unlock();         // ALWAYS unlock in finally
    }
}

// Atomic classes - lock-free, CAS-based thread safety for single variables
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();
counter.compareAndSet(5, 10);
```

Aspect	synchronized	ReentrantLock
Fairness	No fairness guarantee	Optional fair mode (FIFO waiting)
Try/timeout	Not possible	tryLock(), tryLock(timeout)
Interruptible	No	lockInterruptibly()
Release	Automatic (JVM manages)	Manual — must unlock() in finally

10. Deadlock

Deadlock occurs when two or more threads each hold a lock the other needs, and neither releases. Classic recipe: Thread A locks resource 1 then waits for resource 2; Thread B locks resource 2 then waits for resource 1.

- Prevention: always acquire multiple locks in a consistent global order.
- Use tryLock() with a timeout to detect and back off instead of blocking forever.
- Keep critical sections small and avoid calling unknown/overridable code while holding a lock.

11. Quick Interview Recap

- Runnable vs Callable? Runnable.run() returns void and can't throw checked exceptions; Callable.call() returns a value and can throw checked exceptions, used with ExecutorService/Future.
- What does volatile NOT guarantee? Atomicity of compound operations (e.g. increment) — only visibility and ordering.
- Why prefer thread pools over manually created threads? Thread creation is expensive; pools reuse threads, bound resource usage, and provide task queuing and lifecycle management.
- What causes deadlock? Circular wait on locks acquired in inconsistent order across threads.
- Difference between wait() and sleep()? wait() releases the monitor lock and must be called inside synchronized code; sleep() does NOT release any lock and can be called anywhere.